香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

Operating Systems
Lecture 4

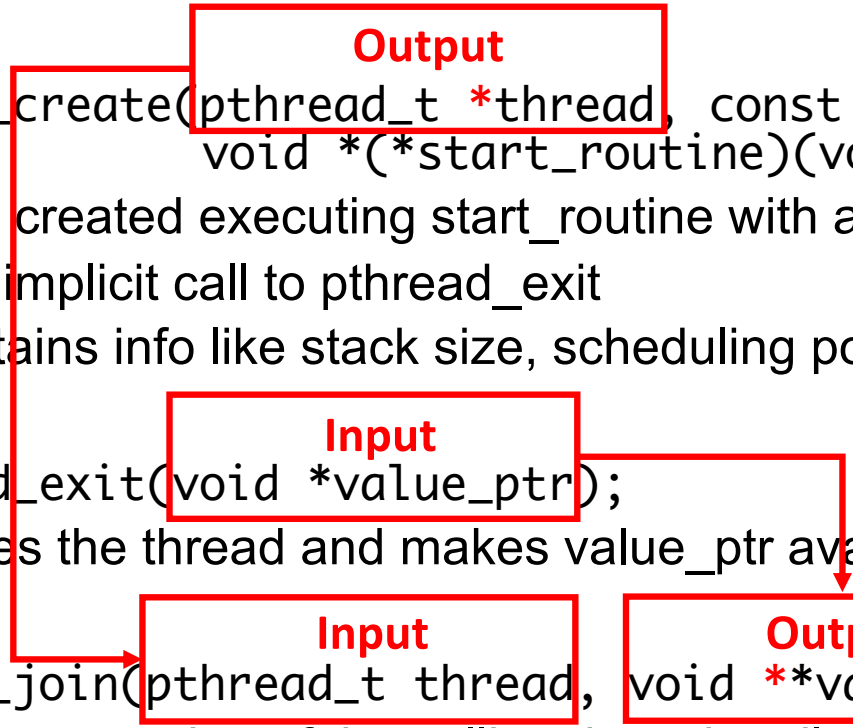
System programming: processes and files

Prof. Yunming XIAO

School of Data Science

Acknowledgments: Ion Stoica and Matei Zaharia, Berkeley CS 162

Recall: OS library API for threads: *pthread*



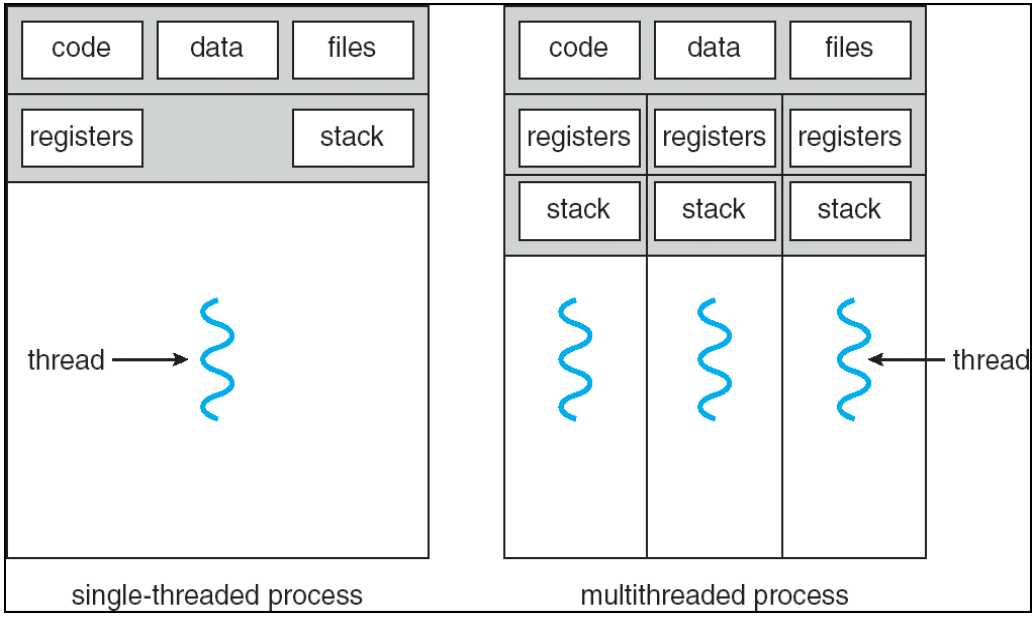
- thread is created executing start_routine with arg as its sole argument.
- return is implicit call to pthread_exit
- (attr contains info like stack size, scheduling policy, etc)

- terminates the thread and makes value_ptr available to any successful join

- suspends execution of the calling thread until the target thread terminates.
- On return with a non-NULL value_ptr the value passed to pthread_exit() by the terminating thread is made available in the location referenced by value_ptr.

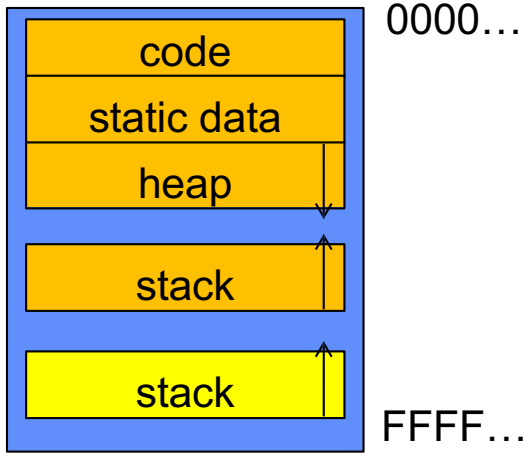
Recall: process

- Processes consists of
  - One or more threads
  - Address space, and other resources shared by the threads within the same process



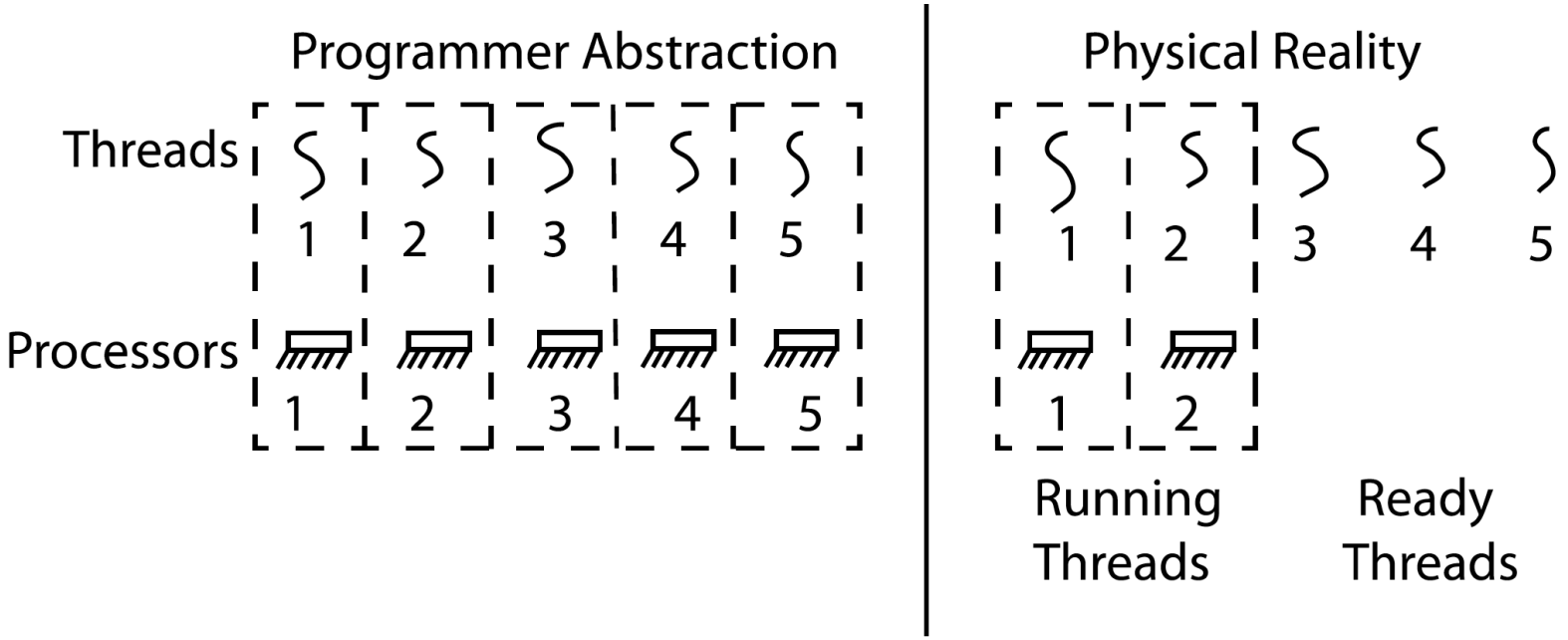
Recall: memory layout with two threads

- Two sets of CPU registers
- Two sets of Stacks
- Issues:
  - How do we position stacks relative to each other?
  - What maximum size should we choose for the stacks?
  - What happens if threads violate this?
  - How might you catch violations?



INTERLEAVING and NONEDETERMINISM
(the beginning of a long discussion)

Thread abstraction

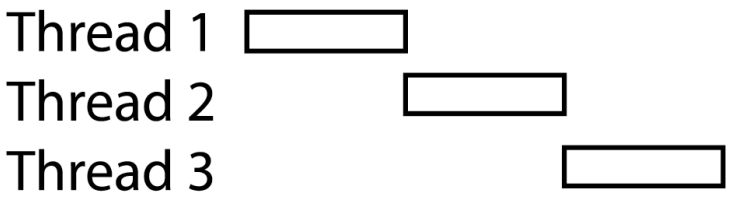


- Illusion: Infinite number of processors
- Reality: threads execute with variable “speed”
  - Programs must be designed to work with any schedule

Programmer's vs. processor's view

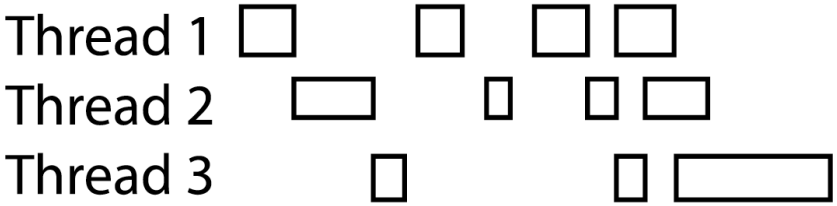
Programmer's View	Possible Execution #1	Possible Execution #2	Possible Execution #3
.	.	.	.
.	.	.	.
.	.	.	.
x = x + 1;	x = x + 1;	x = x + 1	x = x + 1
y = y + x;	y = y + x;	.....	y = y + x
z = x +5y;	z = x + 5y;	thread is suspended	.....
.	.	other thread(s) run	thread is suspended
.	.	thread is resumed	other thread(s) run
.	.	.....	thread is resumed
		y = y + x	.....
		z = x + 5y	z = x + 5y

Possible executions



a) One execution

b) Another execution



c) Another execution

Correctness vs. concurrent threads

- Non-determinism:
  - Scheduler can run threads in any order
  - Scheduler can switch threads at any time
  - This can make testing very difficult
- Independent threads
  - No state shared with other threads
  - Deterministic, reproducible conditions
- Cooperating threads
  - Shared state between multiple threads
- Goal: correctness by design

Race condition: example 1

- Initially x = 0 and y = 0

Thread A	**Thread B**
`x = 1;`	`y = 2;`

- What are the possible values of x below after all threads finish?
  - Must be 1. Thread B does not interfere

Race condition: example 2

- Initially x = 0 and y = 0

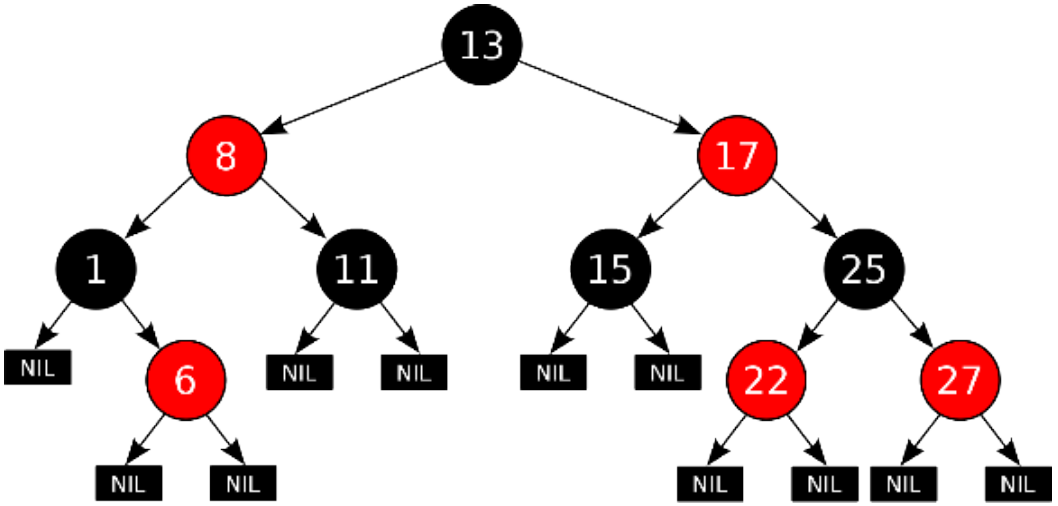
Thread A	**Thread B**
`x = y + 1;`	`y = 2;`
	`y = y * 2;`

- What are the possible values of x below?
  - 1 or 3 or 5 (non-deterministically)
- Race condition: Thread A races against Thread B!

Example: shared data structure

Thread A

```
Insert(3)
```



Tree-Based Set Data Structure

Thread B

```
Insert(4)
Get(6)
```

- How do we make sure this executes correctly?

Relevant definitions

- Synchronization: Coordination among threads, usually regarding shared data

- Mutual Exclusion: Ensuring only one thread does a particular thing at a time (one thread excludes the others)
  - Type of synchronization

- Critical Section: Code exactly one thread can execute at once
  - Result of mutual exclusion

- Lock: An object only one thread can hold at a time
  - Provides mutual exclusion

Locks

- Locks provide two atomic operations:
  - Lock.acquire() – wait until lock is free; then mark it as busy
    - After this returns, we say the calling thread holds the lock
  - Lock.release() – mark lock as free
    - Should only be called by a thread that currently holds the lock
    - After this returns, the calling thread no longer holds the lock

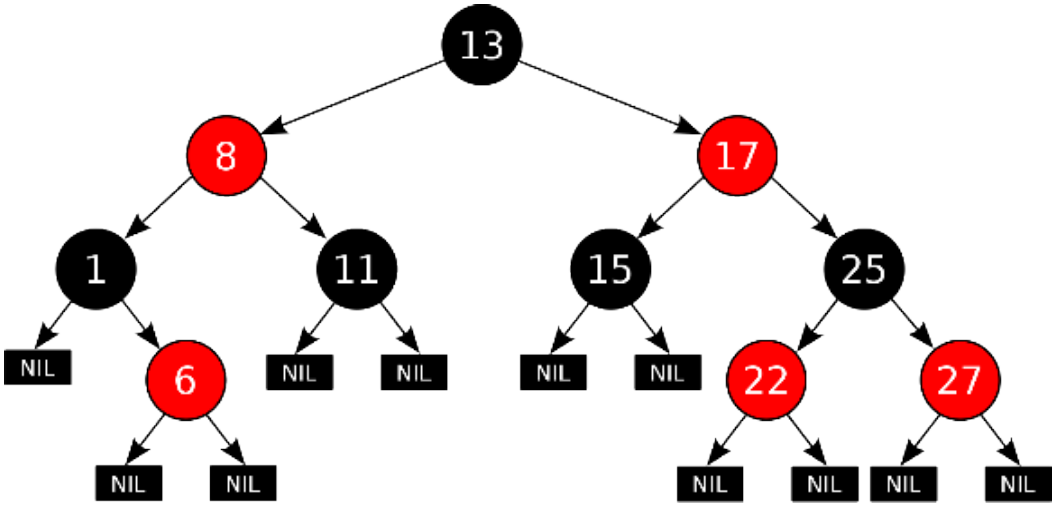
- For now, don't worry about how to implement locks!
  - We'll cover that in substantial depth later on in the class

Example: shared data structure

Thread A

Insert(3)

- lock.acquire()
- Insert 3 into the data structure
- lock.release()



Tree-Based Set Data Structure

Thread B

Insert(4)

- lock.acquire()
- Insert 4 into the data structure
- lock.release()

Get(6)

- lock.acquire()
- Check for membership
- lock.release()

OS library hooks: *pthreads*

```
int pthread_mutex_init(pthread_mutex_t *mutex,
                       const pthread_mutexattr_t *attr)

int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

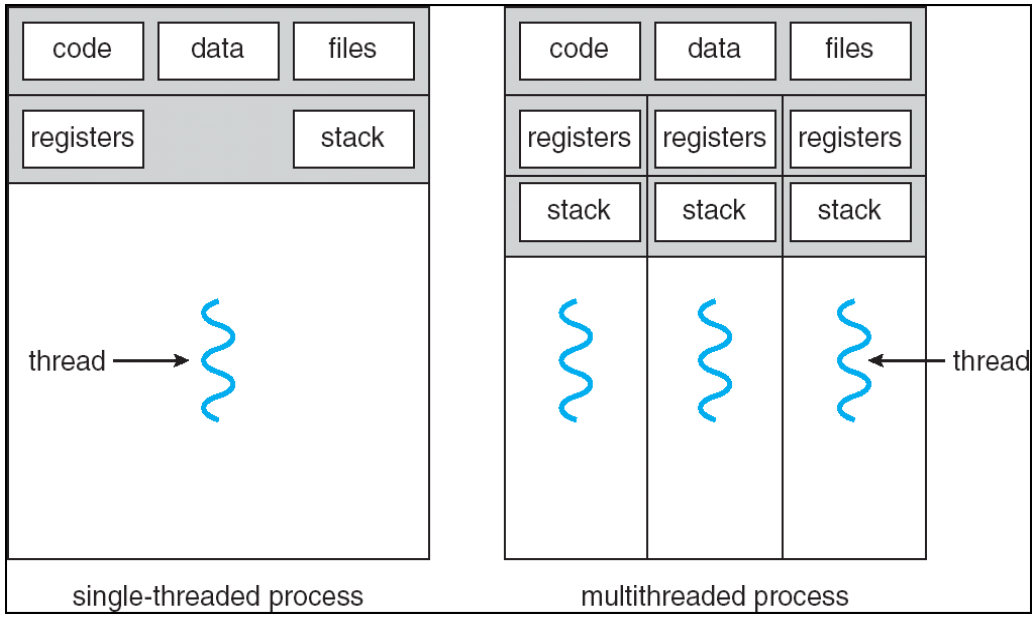
- You'll get a chance to use these in Assignment 1

Processes

Managing processes

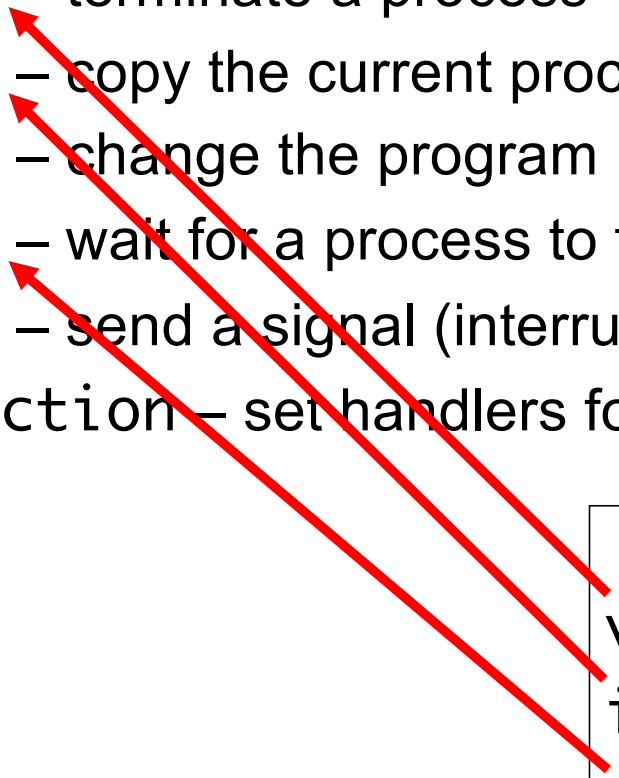
- How to manage process state?
  - How to create a process?
  - How to exit from a process?
- Remember: everything outside of the kernel is running in a process!
- Processes are created and managed by…

  processes!



Process management API

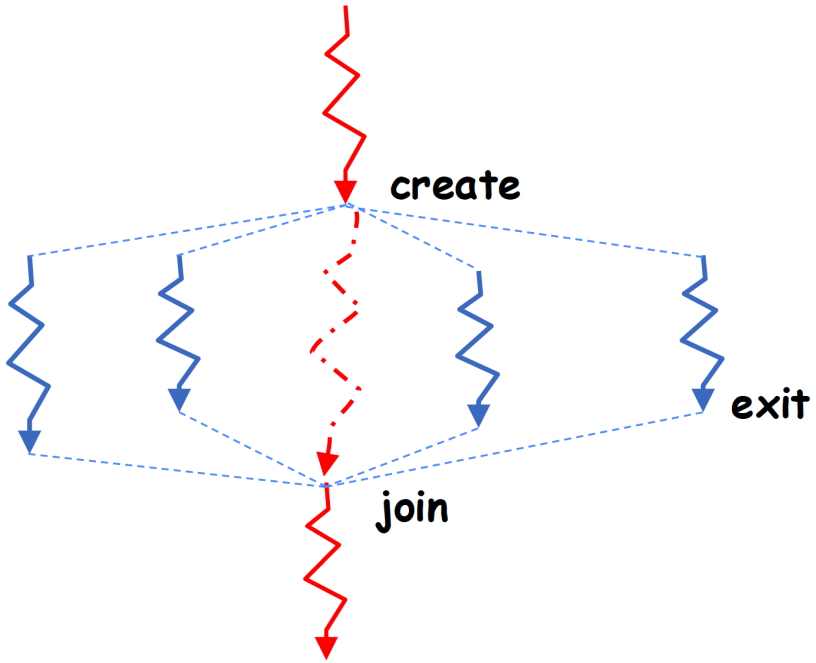
- `exit` – terminate a process
- `fork` – copy the current process
- `exec` – change the program being run by the current process
- `wait` – wait for a process to finish
- `kill` – send a signal (interrupt-like notification) to another process
- `sigaction` – set handlers for signals



Recall threads

```
void pthread_exit
int pthread_create
pthread_join
```

Recall: fork-join pattern



- Main thread *creates* (forks) collection of sub-threads passing them args to work on…
- … and then *joins* with them, collecting results.

Process management API

- `exit` – terminate a process
- `fork` – copy the current process
- `exec` – change the program being run by the current process
- `wait` – wait for a process to finish
- `kill` – send a signal (interrupt-like notification) to another process
- `sigaction` – set handlers for signals

pid.c

```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
   /* get current process' PID */
   pid_t pid = getpid();
   printf("My pid: %d\n", pid);

   exit(0);
}
```

- Q: What if we let main return without ever calling exit?
  - The OS Library calls exit() for us!
  - The entrypoint of the executable is in the OS library
  - OS library calls main
  - If main returns, OS library calls exit

Process management API

- `exit` – terminate a process
- `fork` – copy the current process
- `exec` – change the program being run by the current process
- `wait` – wait for a process to finish
- `kill` – send a signal (interrupt-like notification) to another process
- `sigaction` – set handlers for signals

Creating processes

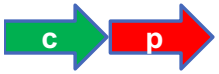
- `pid_t fork()` – copy the current process
  - New process has different pid
  - New process contains a single thread
- Return value from `fork()`: pid (like an integer)
  - When > 0:
    - Running in (original) parent process
    - return value is pid of new child
  - When = 0:
    - Running in new child process
  - When < 0:
    - Error! Must handle somehow
    - Running in original process
- State of original process duplicated in *both* parent and child!
  - Address Space (Memory), File Descriptors (covered later), etc…

fork1.c

```
#include <stdlib.h>  #include <stdio.h>
#include <unistd.h>  #include <sys/types.h>

int main(int argc, char *argv[]) {
  pid_t cpid, mypid;
  pid_t pid = getpid();                    /* get current process' PID */
  printf("Parent pid: %d\n", pid);
  cpid = fork();
  if (cpid > 0) {                          /* parent process */
    mypid = getpid();
    printf("[%d] parent of [%d]\n", mypid, cpid);
  } else if (cpid == 0) {                  /* child process */
    mypid = getpid();
    printf("[%d] child\n", mypid);
  } else {
    perror("Fork failed");
  }
}
```

fork1.c

```
#include <stdlib.h>  #include <stdio.h>
#include <unistd.h>  #include <sys/types.h>

int main(int argc, char *argv[]) {
  pid_t cpid, mypid;
  pid_t pid = getpid();                    /* get current process' PID */
  printf("Parent pid: %d\n", pid);
  cpid = fork();
  if (cpid > 0) {                          /* parent process */
    mypid = getpid();
    printf("[%d] parent of [%d]\n", mypid, cpid);
  } else if (cpid == 0) {                  /* child process */
    mypid = getpid();
    printf("[%d] child\n", mypid);
  } else {
    perror("Fork failed");
  }
}
```



fork1.c

```
#include <stdlib.h>  #include <stdio.h>
#include <unistd.h>  #include <sys/types.h>

int main(int argc, char *argv[]) {
  pid_t cpid, mypid;
  pid_t pid = getpid();                    /* get current process' PID */
  printf("Parent pid: %d\n", pid);
  cpid = fork();
  if (cpid > 0) {                          /* parent process */
    mypid = getpid();
    printf("[%d] parent of [%d]\n", mypid, cpid);
  } else if (cpid == 0) {                  /* child process */
    mypid = getpid();
    printf("[%d] child\n", mypid);
  } else {
    perror("Fork failed");
  }
}
```

p

c

Mystery: fork_race.c

```
int i;
pid_t cpid = fork();
if (cpid > 0) {
  for (i = 0; i < 10; i++) {
    printf("Parent: %d\n", i);
    // sleep(1);
  }
} else if (cpid == 0) {
  for (i = 0; i > -10; i--) {
    printf("Child: %d\n", i);
    // sleep(1);
  }
}
```

- Recall: a process consists of one or more threads executing in an address space
- Here, each process has a single thread
- These threads execute concurrently

- What does this print?
- Would adding the calls to sleep() matter?

Process management API

- `exit` – terminate a process
- `fork` – copy the current process
- `exec` – change the program being run by the current process
- `wait` – wait for a process to finish
- `kill` – send a signal (interrupt-like notification) to another process
- `sigaction` – set handlers for signals

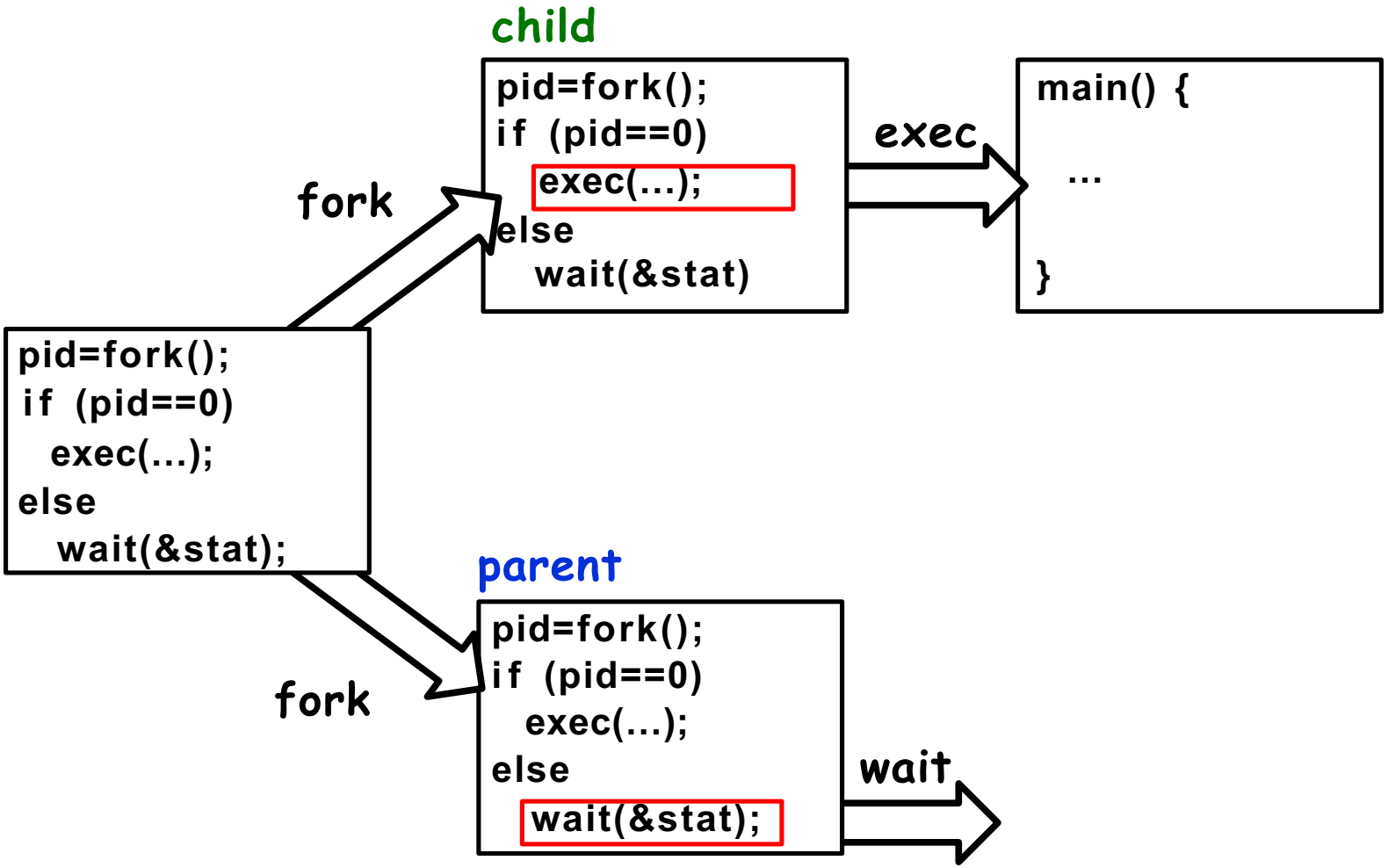
Starting a new program: variants of *exec*

```
...
cpid = fork();
if (cpid > 0) {                          /* parent process */
  tcpid = wait(&status)
  printf("[%d] parent of [%d]\n", mypid, cpid);
} else if (cpid == 0) {                  /* child process */
  char *args[] = {"ls", "-l", NULL};
  execv("/bin/ls", args);

  /* execv doesn't return when it works.
     So, if we got here, it failed! */

  perror("execv");
  exit(1);
}
...
```

fork2.c – parent waits for child to finish

```
int status;
pid_t tcpid;
...
cpid = fork();
if (cpid > 0) {                          /* parent process */
  mypid = getpid();
  printf("[%d] parent of [%d]\n", mypid, cpid);
  tcpid = wait(&status);
  printf("[%d] bye %d(%d)\n", mypid, tcpid, status);
} else if (cpid == 0) {  /* Child Process */
  mypid = getpid();
  printf("[%d] child\n", mypid);
  char *args[] = {"ls", "-l", NULL};
  execv("/bin/ls", args);
  ...
}
...
```

Process management: the Shell pattern



Process management API

- `exit` – terminate a process
- `fork` – copy the current process
- `exec` – change the program being run by the current process
- `wait` – wait for a process to finish
- `kill` – send a signal (interrupt-like notification) to another process
- `sigaction` – set handlers for signals

inf_loop.c

```
#include <stdlib.h>
#include <stdio.h>
#include <signal.h>

void signal_callback_handler(int signum) {
  printf("Caught signal!\n");
  exit(1);
}
int main() {
  struct sigaction sa;
  sa.sa_flags = 0;
  sigemptyset(&sa.sa_mask);
  sa.sa_handler = signal_callback_handler;
  sigaction(SIGINT, &sa, NULL);
  while (1) {}
}
```

- Q: What would happen if the process receives a SIGINT signal, but does not register a signal handler?
  - The process dies!
- For each signal, there is a default handler defined by the system

Process management API

- `SIGINT` – control-C
- `SIGTERM` – default for shell command `kill`
- `SIGSTOP` – control-Z (default action: stop process)

- `SIGKILL`, `SIGSTOP` – wait for a process to finish
  - Can't be changed with `sigaction`
  - Why?

Process vs. thread API

- Why have `fork()` and `exec()` system calls for processes, but just a `pthread_create()` function for threads?
  - Convenient to `fork` without `exec`: put code for parent and child in one executable instead of multiple
  - It will allow us to programmatically control child process' state
    - By executing code before calling `exec()` in the child
  - We'll see this in the case of File I/O later

- Windows uses `CreateProcess()` instead of `fork()`
  - Also works, but a more complicated interface

Group discussion

- Topic: processes vs. threads
  - If we have two tasks to run concurrently, do we run them in separate threads, or do we run them in separate processes?
  - What are the pros and cons?

- Discuss in groups of two to three students
  - Each group chooses a leader to summarize the discussion
  - In your group discussion, please do not dominate the discussion, and give everyone a chance to speak

(Optional) runtime-managed concurrency: illusion of threads

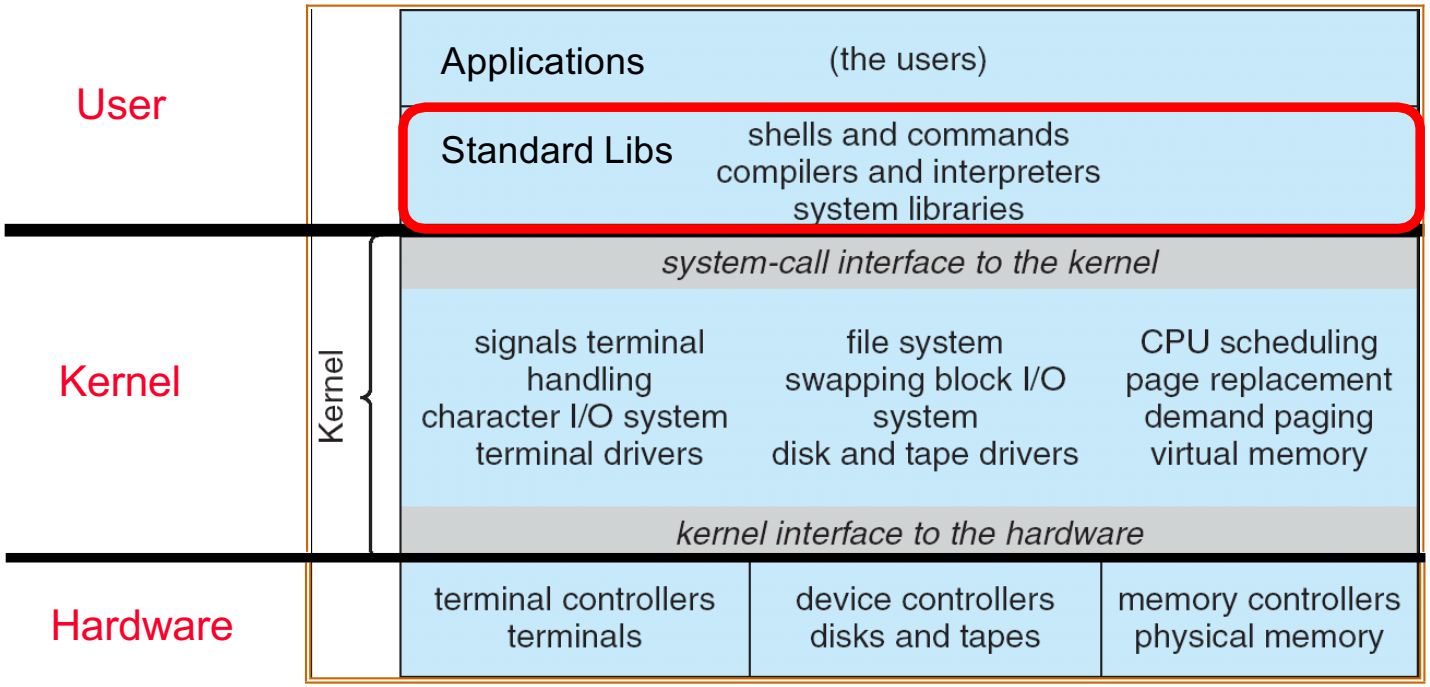
- Are processes and threads enough? Do we need a different design with different trade-offs?

- Languages like Golang introduces goroutines
- This adds a runtime layer between your code and OS threads
- This layer controls scheduling and switching, instead of the OS!
- Trade-off: simplicity and scale in exchange for direct OS control

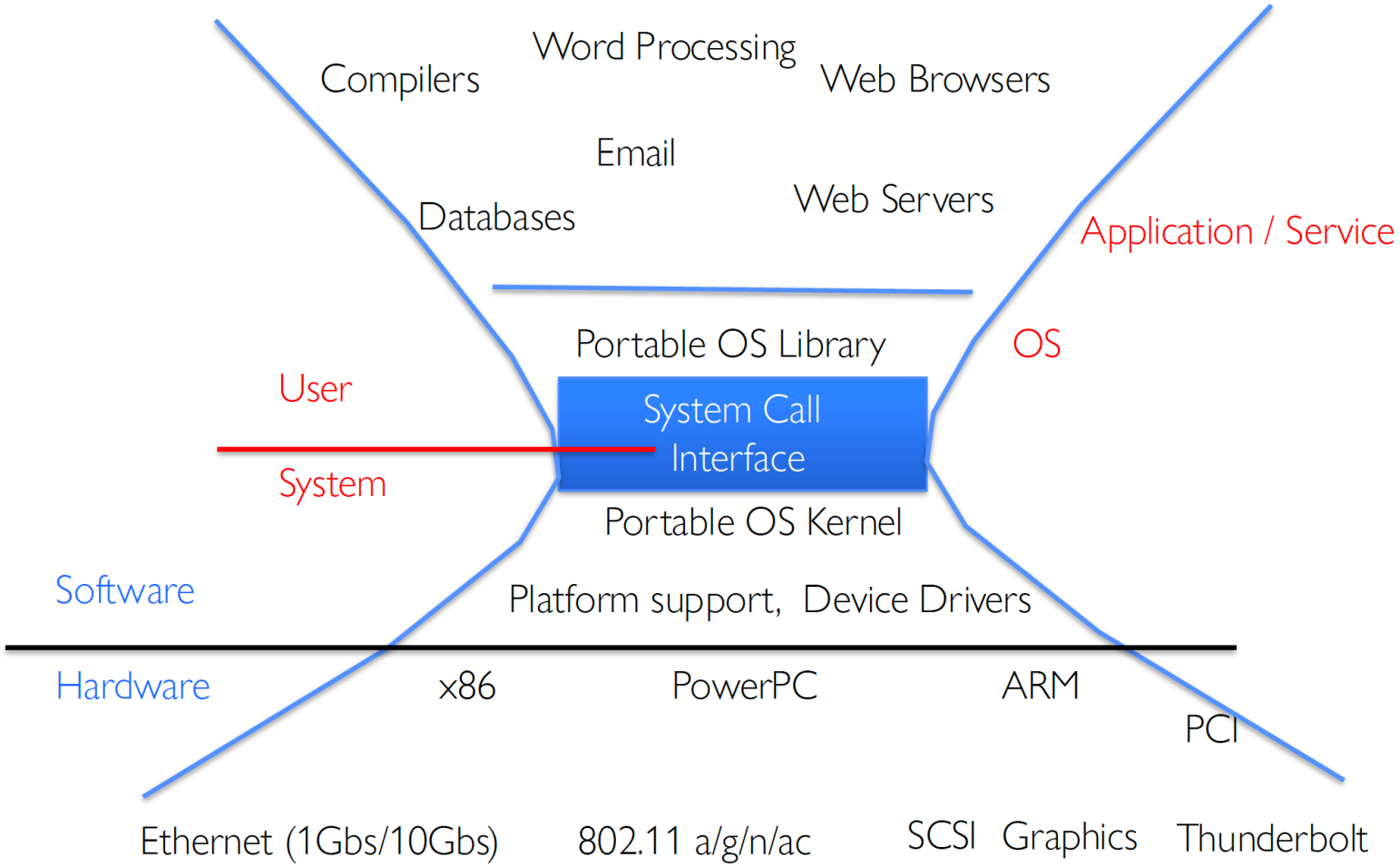
- Read more here: https://omjogani.github.io/blogs/posts/the-ultimate-guide-to-concurrency-with-go-routines/

Files

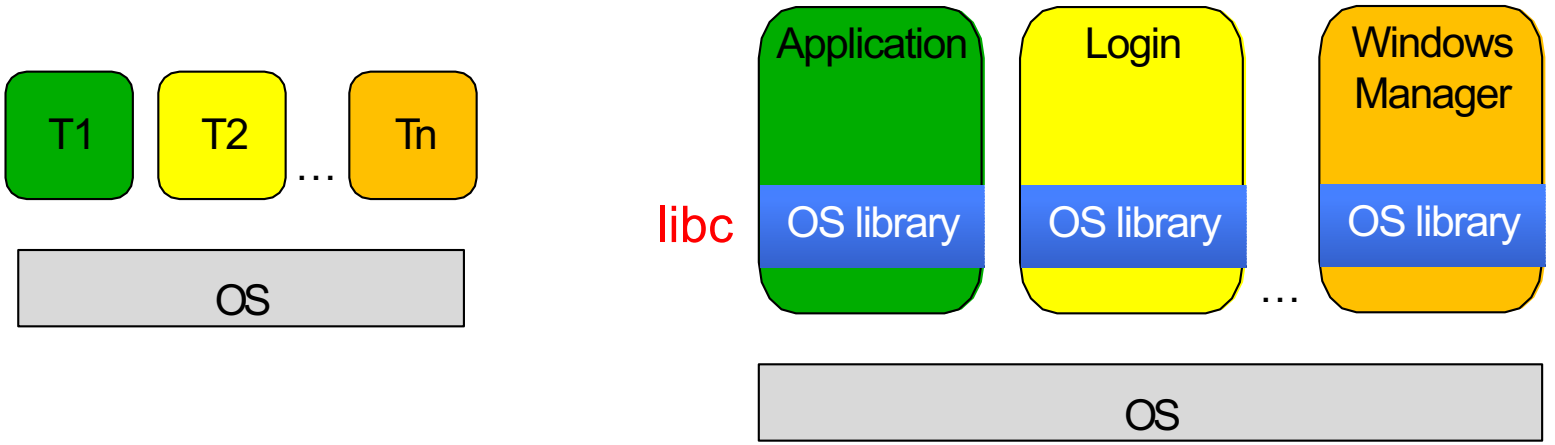
Recall: UNIX system structure



A kind of narrow waist



Recall: OS library (libc) issues syscalls



- OS Library: Code linked into the user-level application that provides a clean or more functional API to the user than just the raw syscalls
  - Most of this code runs at user level, but makes syscalls (which run at kernel level)

UNIX/POSIX idea: everything is a “file”

- Almost identical interface for:
  - Files on disk
  - Devices (terminals, printers, etc.)
  - Regular files on disk
  - Networking (sockets)
  - Local interprocess communication (pipes, sockets)
- Based on the system calls `open()`, `read()`, `write()`, & `close()`
- Additional: `ioctl()` for custom configuration that doesn’t quite fit
- Note that the “Everything is a File” idea was a radical idea when proposed
  - Dennis Ritchie and Ken Thompson described this idea in their seminal paper on UNIX called “The UNIX Time-Sharing System” from 1974
  - Available online

Aside: POSIX interfaces

- POSIX: Portable Operating System Interface (for uniX?)
  - Interface for application programmers (mostly)
  - Defines the term “Unix,” derived from AT&T Unix
  - Created to bring order to many Unix-derived OSes, so applications are portable
    - Partially available on non-Unix OSes, like Windows
  - Requires standard system call interface

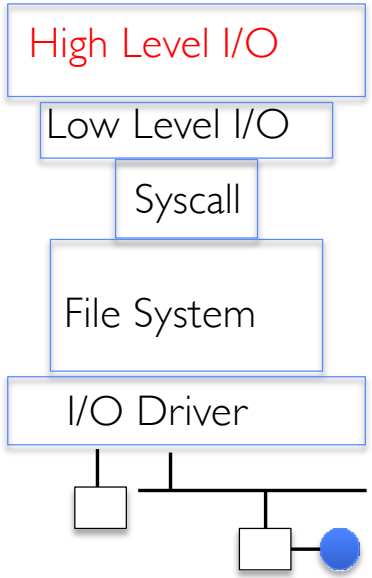
The file system abstraction

- File
  - Named collection of data in a file system
  - POSIX File data: sequence of bytes
    - Could be text, binary, serialized objects, …
  - File Metadata: information about the file
    - Size, Modification Time, Owner, Security info, Access control
- Directory
  - “Folder” containing files & directories
  - Hierarchical naming
    - Path through the directory graph
    - Uniquely identifies a file or directory
  - Links and Volumes (later)

Connecting processes, file systems, and users

- Every process has a *current working directory* (CWD)
  - Can be set with system call:
    ```
    int chdir(const char *path); //change CWD
    ```
- Absolute paths ignore CWD
  - /home/oski/csc3150
- Relative paths are relative to CWD
  - index.html, ./index.html
    - Refers to index.html in current working directory
  - ../index.html
    - Refers to index.html in parent of current working directory
  - ~/index.html, ~csc3150/index.html
    - Refers to index.html in the home directory

I/O and storage layers



Streams (buffered I/O)

File Descriptors

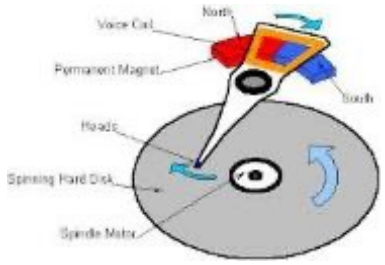
open(), read(), write(), close(), …

Open File Descriptions

Files/Directories/Indexes

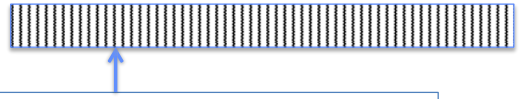
Commands and Data Transfers

Disks, Flash, Controllers, DMA



C high-level file API – streams

- Operates on "streams" – unformatted sequences of bytes (wither text or binary data), with a position:



```
#include <stdio.h>
FILE *fopen( const char *filename, const char *mode );
int fclose( FILE *fp );
```

Mode Text	Binary	Descriptions
r	rb	Open existing file for reading
w	wb	Open for writing; created if does not exist
a	ab	Open for appending; created if does not exist
r+	rb+	Open existing file for reading & writing.
w+	wb+	Open for reading & writing; truncated to zero if exists, create otherwise
a+	ab+	Open for reading & writing. Created if does not exist. Read from beginning, write as append

- Open stream represented by pointer to a FILE data structure
  - Error reported by returning a NULL pointer

C API standard stream – stdio.h

- Three predefined streams are opened implicitly when the program is executed.
  - `FILE *stdin` – normal source of input, can be redirected
  - `FILE *stdout` – normal source of output, can too
  - `FILE *stderr` – diagnostics and errors

- `STDIN / STDOUT` enable composition in Unix

- All can be redirected
  - `cat hello.txt | grep "World!"`
  - **`cat`**'s **`stdout`** goes to **`grep`**'s **`stdin`**

C high-level file API

```
// character oriented
int fputc( int c, FILE *fp );                 // return c or EOF on err
int fputs( const char *s, FILE *fp );         // return > 0 or EOF

int fgetc( FILE * fp );
char *fgets( char *buf, int n, FILE *fp );

// block oriented
size_t fread(void *ptr, size_t size_of_elements,
             size_t number_of_elements, FILE *a_file);
size_t fwrite(const void *ptr, size_t size_of_elements,
size_t number_of_elements, FILE *a_file);

// formatted
int fprintf(FILE *restrict stream, const char *restrict format, ...);
int fscanf(FILE *restrict stream, const char *restrict format, ... );
```

C streams: char-by-char I/O

```
int main(void) {
  FILE* input = fopen(“input.txt”, “r”);
  FILE* output = fopen(“output.txt”, “w”);
  int c;

  c = fgetc(input);
  while (c != EOF) {
    fputc(output, c);
    c = fgetc(input);
  }
  fclose(input);
  fclose(output);
}
```

C high-level file API

```
// character oriented
int fputc( int c, FILE *fp );                 // return c or EOF on err
int fputs( const char *s, FILE *fp );         // return > 0 or EOF

int fgetc( FILE * fp );
char *fgets( char *buf, int n, FILE *fp );

// block oriented
size_t fread(void *ptr, size_t size_of_elements,
             size_t number_of_elements, FILE *a_file);
size_t fwrite(const void *ptr, size_t size_of_elements,
size_t number_of_elements, FILE *a_file);

// formatted
int fprintf(FILE *restrict stream, const char *restrict format, ...);
int fscanf(FILE *restrict stream, const char *restrict format, ... );
```

C streams: block-by-block I/O

```
#define BUFFER_SIZE 1024
int main(void) {
  FILE* input = fopen("input.txt", "r");
  FILE* output = fopen("output.txt", "w");
  char buffer[BUFFER_SIZE];
  size_t length;
  length = fread(buffer, sizeof(char), BUFFER_SIZE, input);
  while (length > 0) {
    fwrite(buffer, sizeof(char), length, output);
    length = fread(buffer, sizeof(char), BUFFER_SIZE, input);
  }
  fclose(input);
  fclose(output);
}
```

Aside: check your errors!

- Systems programmers should always be paranoid!
  - Otherwise, you get intermittently buggy code
- We should really be writing things like:

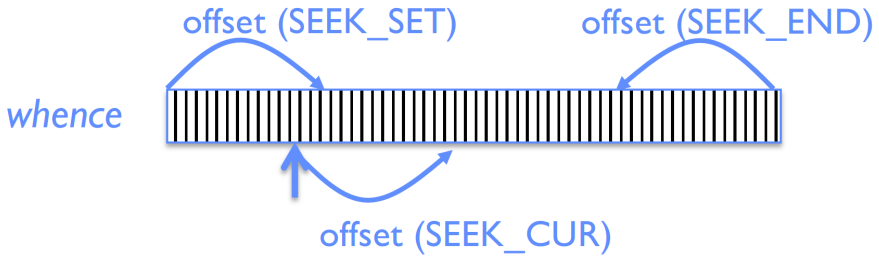
```
FILE* input = fopen("input.txt", "r");
if (input == NULL) {
  // Prints our string and error msg
  perror("Failed to open input file");
}
```

- Be **thorough about checking return values!**
  - Want failures to be systematically caught and dealt with
- I may be a bit loose with error checking for examples in class (to keep short)
  - Do as I say, not as I show in class!

C high-level file API: positioning the pointer

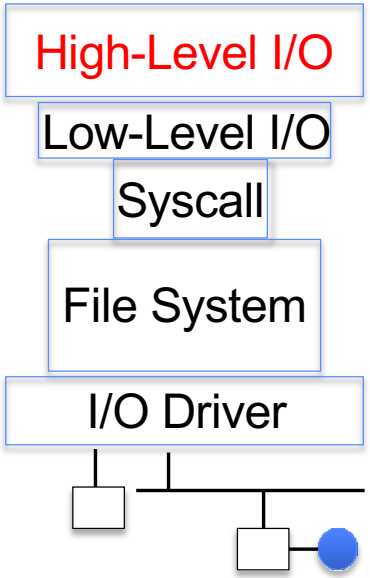
```
int fseek(FILE *stream, long int offset, int whence)
long int ftell(FILE *stream)
void rewind(FILE *stream)
```

- For `fseek()`, the `offset` is interpreted based on the `whence` argument (constants in stdio.h):
  - `SEEK_SET`: Then offset interpreted from beginning (position 0)
  - `SEEK_END`: Then offset interpreted backwards from end of file
  - `SEEK_CUR`: Then offset interpreted from current position



- Overall preserves high-level abstraction of a uniform stream of objects

I/O and storage layers



Streams (buffered I/O)

File Descriptors

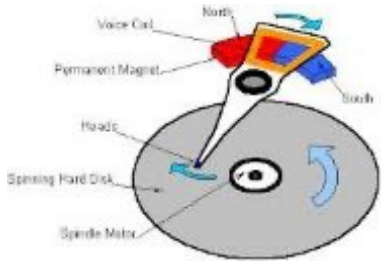
open(), read(), write(), close(), …

Open File Descriptions

Files/Directories/Indexes

Commands and Data Transfers

Disks, Flash, Controllers, DMA



Low-level file I/O: the RAW system-call interface

```
#include <fcntl.h>  #include <unistd.h>  #include <sys/types.h>
int open (const char *filename, int flags [, mode_t mode])
int creat (const char *filename, mode_t mode)
int close (int filedes)
```

Bit vector of:
- Access modes (Rd,Wr, …)
- Open Flags (Create, …)
- Operating modes (Appends, …)

Bit vector of Permission Bits:
- User|Group|Other X R|W|X

- Integer return from open() is a *file descriptor*
  - Error indicated by return < 0: the global `errno` variable set with error (see man pages)
- Operations on file descriptors:
  - Open system call create an open file description entry in system-wide table of open files
  - Open file description object in the kernel represents an instance of an open file
  - Why give user an integer instead of a pointer to the file description in kernel?

C low-level (pre-opened) standard descriptors

```
#include <unistd.h>
STDIN_FILENO  – macro has value 0
STDOUT_FILENO – macro has value 1
STDERR_FILENO – macro has value 2

// Get file descriptor inside FILE *
int fileno (FILE *stream)

// Make FILE * from descriptor
FILE * fdopen (int filedes, const char *opentype)
```

Low-level file API

- Read data from open file using file descriptor:

```
ssize_t read (int filedes, void *buffer, size_t maxsize)
```

  - Reads up to maxsize bytes – might actually read less!
  - returns bytes read, 0 => EOF, -1 => error

- Write data to open file using file descriptor

```
ssize_t write (int filedes, const void *buffer, size_t size)
```

  - returns number of bytes written

- Reposition file offset within kernel (this is independent of any position held by high-level FILE descriptor for this file!

```
off_t lseek (int filedes, off_t offset, int whence)
```

Example: lowio.c

```
int main() {
  char buf[1000];
  int     fd = open("lowio.c", O_RDONLY);
  ssize_t rd = read(fd, buf, sizeof(buf));
  int     err = close(fd);
  ssize_t wr = write(STDOUT_FILENO, buf, rd);
}
```

- How many bytes does this program read?

POSIX I/O: design patterns

- Open before use
  - Access control check, setup happens here
- Byte-oriented
  - Least common denominator
  - OS responsible for hiding the fact that real devices may not work this way (e.g. hard drive stores data in blocks)
- Explicit close

POSIX I/O: kernel buffering

- Reads are buffered inside kernel
  - Part of making everything byte-oriented
  - Process is blocked while waiting for device
  - Let other processes run while gathering result
- Writes are buffered inside kernel
  - Complete in background (more later on)
  - Return to user when data is “handed off” to kernel

- This buffering is part of global buffer management and caching for block devices (such as disks)
  - Items typically cached in quanta of disk block sizes
  - We will have many interesting things to say about this buffering when we dive into the kernel

Low-level I/O: other operations

- Operations specific to terminals, devices, networking, …
  - e.g., `ioctl`
- Duplicating descriptors
  - `int dup2(int old, int new);`
  - `int dup(int old);`
- Pipes – channel
  - `int pipe(int pipefd[2]);`
  - Writes to `pipefd[1]` can be read from `pipefd[0]`
- File Locking
- Memory-Mapping Files
- Asynchronous I/O

Low-level vs. high-level file API

- Low-level direct use of syscall interface:
  open(), read(), write(), close()
- Opening of file returns file descriptor:
  int myfile = open(…);
- File descriptor only meaningful to kernel
  - Index into process (PDB) which holds pointers to kernel-level structure (“file description”) describing file.
- Every read() or write() causes syscall no matter how small (could read a single byte)
- Consider loop to get 4 bytes at a time using read():
  - Each iteration enters kernel for 4 bytes.

- High-level buffered access:
  fopen(), fread(), fwrite(), fclose()
- Opening of file returns ptr to FILE:
  FILE *myfile = fopen(…);
- FILE structure is user space contains:
  - a chunk of memory for a buffer
  - the file descriptor for the file (fopen() will call open() automatically)
- Every fread() or fwrite() filters through buffer and may not call read() or write() on every call.
- Consider loop to get 4 bytes at a time using fread():
  - First call to fread() calls read() for block of bytes (say 1024). Puts in buffer and returns first 4 to user.
  - Subsequent fread() grab bytes from buffer

Low-level vs. high-level file API

- Low-level operations

```
ssize_t read(…) {
    asm code … syscall # into %eax
    put args into registers %ebx, …
    special trap instruction
        Kernel:
            get args from regs
            dispatch to system func
            do the work to read from the file
            store return value in %eax
    get return values from reg
    return data to caller
}
```

- High-level operations

```
ssize_t fread(…) {
    check buffer for contents
    return data to caller if available
    asm code … syscall # into %eax
    put args into registers %ebx, …
    special trap instruction
        Kernel:
            get args from regs
            dispatch to system func
            do the work to read from the file
            store return value in %eax
    get return values from reg
    update buffer with excess data
    return data to caller }
```

Low-level vs. high-level file API

- Low-level operations on file descriptors are visible immediately

```
write(STDOUT_FILENO, "Beginning of line ", 18);
sleep(10);
write("and end of line \n", 16);
```

  - Outputs "Beginning of line" 10 seconds earlier than “and end of line”

- High-level streams are buffered in user memory:

```
printf("Beginning of line ");
sleep(10); // sleep for 10 seconds
printf("and end of line\n");
```

  - Prints out everything at once

Group discussion

- Topic: buffer in user space
  - Why buffer in user space? Why not?
  - What are the pros and cons?

- Discuss in groups of two to three students
  - Each group chooses a leader to summarize the discussion
  - In your group discussion, please do not dominate the discussion, and give everyone a chance to speak

Why buffer in userspace? Overhead!

- Syscalls are more expensive than function calls
- `read`/`write` a file byte by byte? Max throughput of ~10MB/second
- With `fgetc`? Keeps up with your SSD

Why buffer in userspace? Functionality!

- System call operations less capable
  - Simplifies kernel

- Example: No “read until new line” operation in kernel
  - Why? Kernel agnostic about formatting!
  - Solution: Make a big read syscall, find first new line in userspace
    - i.e. use one of the following high-level options:

```
char *fgets(char *s, int size, FILE *stream);
ssize_t getline(char **lineptr, size_t *n, FILE *stream);
```

Conclusion

- System call interface is “narrow waist” between user programs and kernel
  - Must enter kernel atomically by setting PC to kernel routine at same time that CPU enters kernel mode
- Processes consist of one or more threads in an address space
  - Abstraction of the machine: execution environment for a program
  - Can use fork, exec, etc. to manage threads within a process
- We saw the role of the OS library
  - Provide API to programs
  - Interface with the OS to request services
- Streaming IO: modeled as a stream of bytes
  - Most streaming I/O functions start with “f” (like “fread”)
  - Data buffered automatically by C-library function
- Low-level I/O:
  - File descriptors are integers
  - Low-level I/O supported directly at system call level

Thanks